

Estructura de Datos

Guía de laboratorio N°4 - Estructuras y Makefile

Milton Fabricio Hernández Morales

Objetivo

El objetivo de esta guía es que revises, antes de la próxima ayudantía, un pequeño proyecto en C que ya usa **estructuras**, para que entiendas cómo está organizado y por qué está hecho de esa forma.

¿Qué tienes que hacer?

Junto a esta guía te dejo un archivo `esqueleto-semana3.zip` (disponible en **Pregrado-virtual**) con un proyecto ya armado, organizado así:

- `src/main.c`: el programa principal. Crea un arreglo de personas, las imprime por pantalla, genera un archivo CSV con esa información y libera la memoria usada.
- `src/persona.c`: las funciones relacionadas con la estructura `Persona` (crearla, imprimirla y eliminarla).
- `inc/person.h`: la definición de la estructura `Persona` y la declaración de sus funciones.
- `build/`: carpeta vacía, donde va a quedar el ejecutable cuando compiles.

Tu tarea es **leer el código con calma** y compilarlo para asegurarte de que funciona en tu computador. Para compilar y ejecutar el programa, ve a la carpeta del proyecto y ejecuta el comando:

```
1 gcc src/* -I inc/ -o build/person.out && ./build/person.out
```

Si todo compila bien, deberías ver los datos de tres personas impresos por pantalla y un archivo `personas.csv` generado dentro de `build/`.

TIP

Compílo **antes** de venir a la ayudantía, no el mismo día de la clase. Así, si te sale algún error (por ejemplo por el compilador o el sistema operativo que uses), tenemos tiempo de revisarlo con calma y no perdemos el tiempo de la clase en eso.

¿En qué tienes que fijarte?

No basta con que el código compile: la idea es que entiendas **qué hace cada parte y por qué está escrita así**. Mientras lo revisas, te dejo algunas preguntas para guiar tu lectura. No es necesario que las respondas por escrito, pero sí que llegues a la ayudantía con una opinión (o con la duda) sobre cada una:

- ¿Por qué el campo `nombre` de `Persona` es un `char*` (un puntero) y no, por ejemplo, un arreglo de tamaño fijo como `char nombre[50]`?
- En `crear_persona`, ¿por qué se le pide memoria a `malloc` solo para `nombre`, y no para `edad` ni para el resto de la estructura?
- ¿Qué pasa si llamas a `crear_persona` con una edad negativa? ¿Por qué crees que se decidió manejarlo de esa forma?
- `eliminar_persona` solo hace `free(p.nombre)`. ¿Por qué no hace falta liberar también `p`?
- ¿Por qué las funciones (`crear_persona`, `print_persona`, etc.) están declaradas en `person.h` pero implementadas en `persona.c`? ¿Qué pasaría si estuvieran todas juntas en un solo archivo?
- Al principio de `person.h` aparece un bloque `#ifndef PERSON` / `#define PERSON` / `#endif`. ¿Para qué sirve?